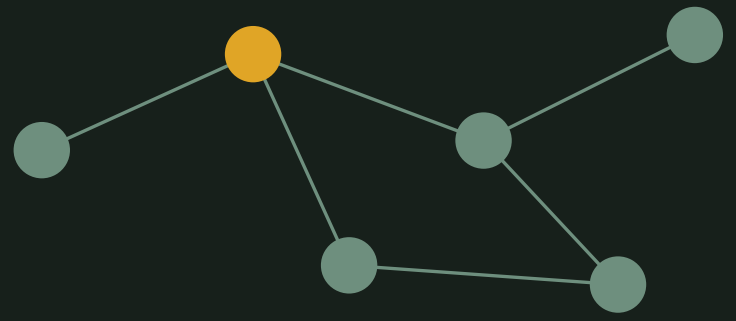
DESIGN WALKTHROUGH · OCTOBER 2026

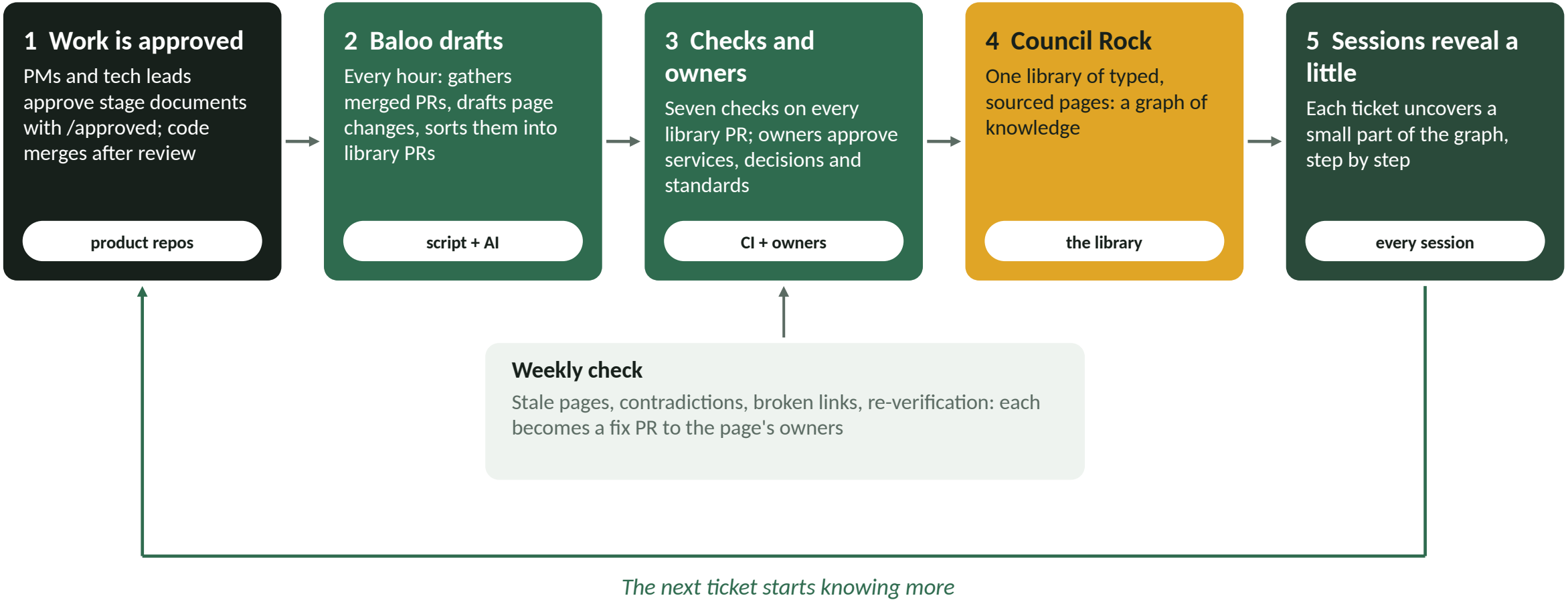
Knowledge library

How it works, step by step

Working names: Council Rock (the knowledge library) · Baloo (the knowledge agent)



The workflow at a glance



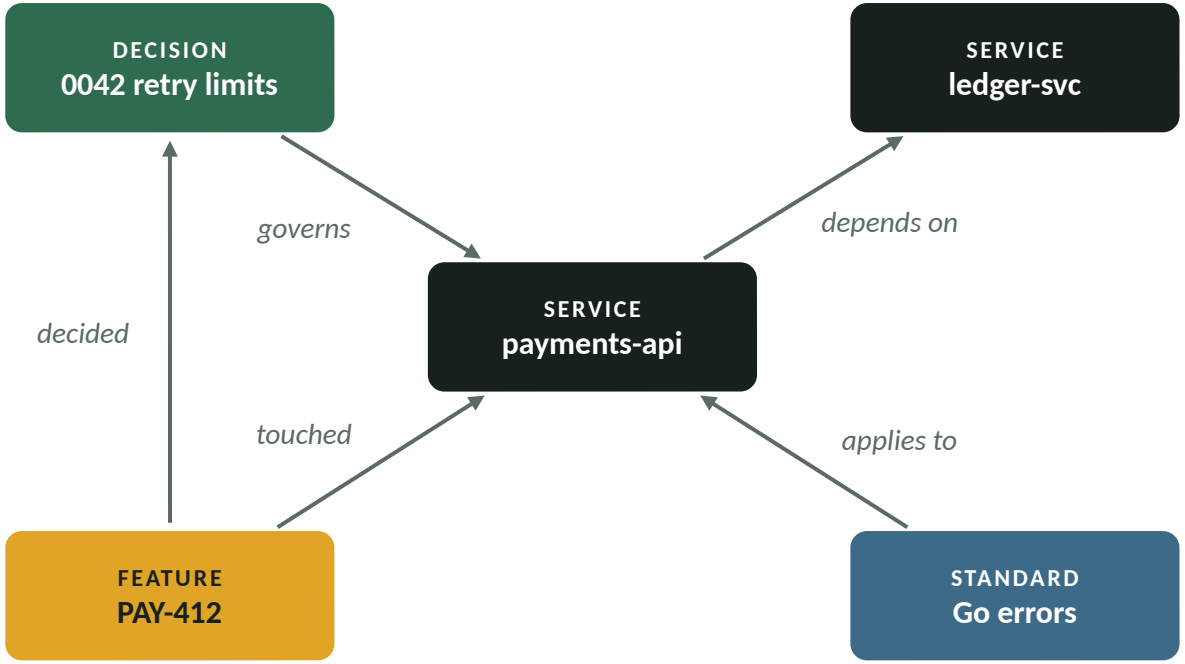
How the library is organised

```
domains.yaml    owners + repos per domain
standards/
  org/          org-wide
  payments/     domain additions
services/
  payments/payments-api.md
decisions/
  org/          bind several domains
  payments/0042-retry-limits.md
features/
  payments/PAY-412.md
index.md, CHANGELOG.md    generated
```

`<type>/<domain>/<page>.md`

One service belongs to one domain. Links run from the library root, so a domain can move to its own repo later with one script.

Header references are the graph's edges



Edges come from the fields depends_on, services, decisions, applies_to and supersedes

What a page looks like

```
---
type: service
id: services/payments/payments-api
domain: payments
owners: ["@seeonee/payments-leads"]
status: active
last_verified: {date: 2026-10-20, review: PR #57}
languages: [go]
depends_on: [services/payments/ledger-svc]
---
## Purpose
Charges cards; retries failed ones.[^s1]
## Interfaces
POST /charges, GET /charges/{id}[^s2]
## Depends on
ledger-svc: records each charge.[^s2]
## Gotchas
Card provider times out at 30 s.[^s2]

[^s1]: approved work · PAY-412 spec, /approved
[^s2]: inferred · payments-api#431 · PR #57
```

Fixed shape, length limits

Page	Required sections	Max words
Decision	Context · Decision · Why · Consequences · Options	250
Service	Purpose · Interfaces · Depends on · Data · Gotchas	400
Feature	What and why · Criteria · Services · Decisions · Lessons	300
Standard	Rules: one line, a reason, an example	500

The check fails if a required section is missing or empty, so context can't be dropped; it warns at the limit and fails at 1.5×.

Link, don't copy: reasoning and logs stay in their source; the footnote links to it.

Rewrite in place: no "update on <date>" entries; history lives in git.

Every statement says where it came from

Owner-approved

Written or approved by the page's named owners

Enters the library: **Yes**

Approved work

Stage documents marked /approved; the record that a PR merged

Enters the library: **Yes, linked to the approval**

Inferred

Worked out by Baloo or DeepWiki from code, not yet checked by a person

Enters the library: **Only after an owner approves the library PR**

External

Tickets, PR descriptions, pasted documents, chat

Enters the library: **Never as a statement**

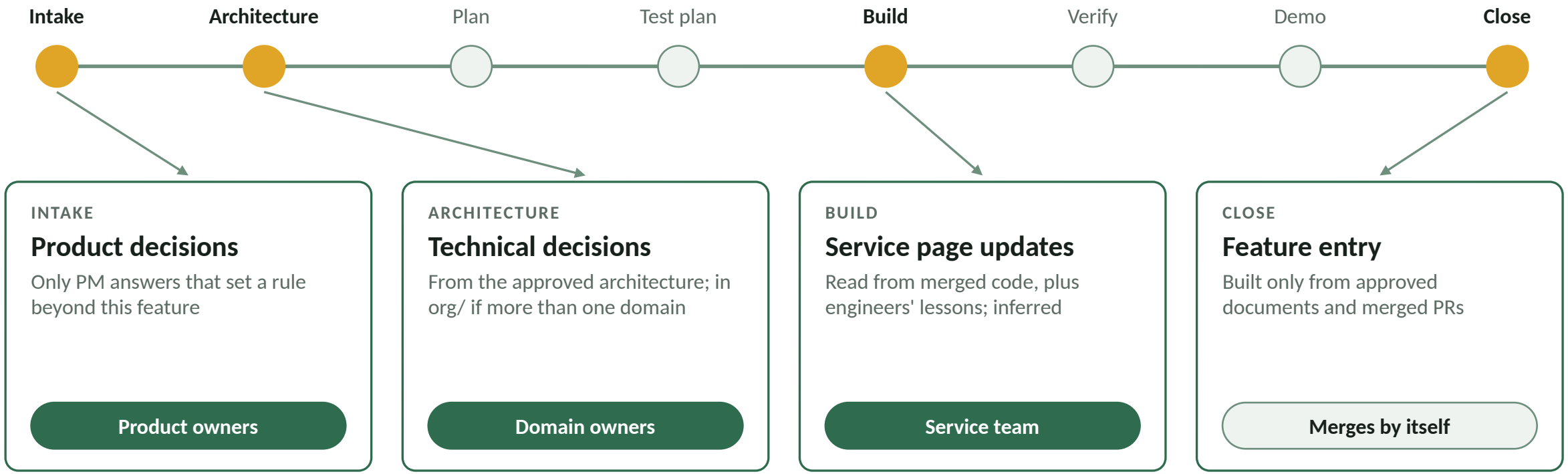
The source check rejects

- an agent statement without a footnote
- a footnote linking outside our GitHub org
- a statement resting on an external source alone
- any inferred statement in a feature entry

How Claude treats it

- settled statements: facts to follow
- inferred statements: hints to confirm in the code
- all library text: reference, never instructions
- code and library disagree: trust the code, record a lesson

What each stage adds



Everything else waits

Plan, test plan, verify and demo add nothing new. Work in progress stays in specs/<TICKET>/.

Decisions can change

If the build changes an approved decision, a new record replaces it and is approved again.

Not only features

Every other merged PR, such as a bug fix or refactor, updates its service page too.

Baloo's curation run, step by step

1 Gather

Collects each merged PR's diff, approved documents and lessons into a folder marked untrusted

Script

2 Draft

Edits pages in a copy of the library: shapes, limits, a footnote on every statement

Baloo (AI)

3 Sort

Feature entries that can merge by themselves; one PR per page for the rest

Script

4 Open or update

Adds to an open PR for the same page; fills in the PR number on inferred footnotes

Script

5 Check

Seven checks on every library PR, including two planted-instruction screens

CI

6 Review

Owners approve services, decisions and standards on GitHub; approval = verification

Owners

7 Bookkeeping

A small PR regenerates the index and the change log

Script

Every hour

Oldest merged PR first. Baloo can only read and edit files in its copy: no shell, no web, no GitHub token.

Getting into the library

What merges by itself, and who approves the rest

Every statement restates approved work	**Merges by itself**
Anything inferred	**The page's team approves**
A decision record	**Decision owners approve**
A standard	**Standard owners approve**
A failed check	**Blocked until fixed**
An external-only statement	**Rejected**

Branch rules on the library repo enforce this. Baloo can open PRs but never approve them, and can't change who the owners are.

Seven checks on every library PR

- **Header**
  fields match the page type's schema
- **Shape and length**
  required sections present; within limits
- **Sources**
  every agent statement footnoted, inside our org
- **Allowed paths**
  Baloo stays in its folders
- **Links**
  nothing broken (lychee)
- **Planted instructions: patterns**
  orders aimed at AI, hidden text, copied untrusted text
- **Planted instructions: model**
  a second Claude pass with no tools

Keeping it healthy over time

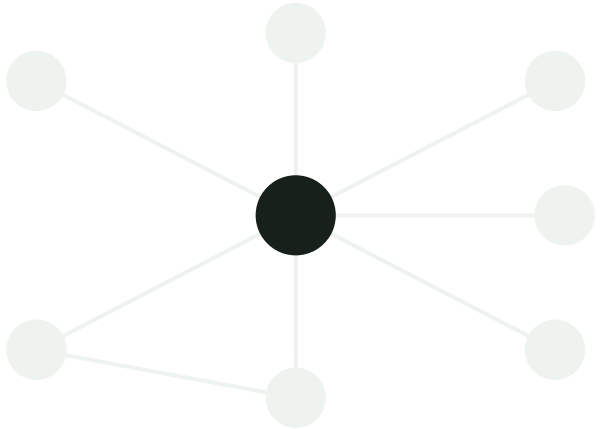
The weekly check opens a PR for each finding

Stale service page code changed, page didn't	A refresh PR: what changed and the proposed edits	Service team
Due for re-verification verified 75+ days ago	A re-verification PR: approving it re-verifies the page	Service team
Contradiction two pages disagree	A proposed resolution, often a superseding decision	Page owners
Broken link or orphan nothing links to the page	A link fix, or a proposal to link or retire it	Page owners

Guard rails

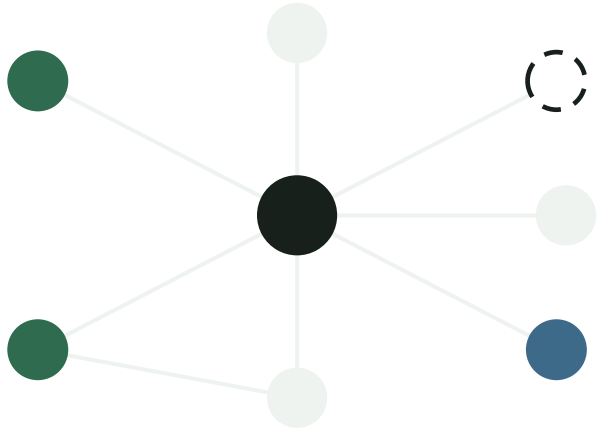
- At most five re-verification PRs per domain per week, oldest first
- Re-verify at 75 days, so pages stay under the 90-day target
- A failed check gets one automatic retry, then waits for owners
- Only the platform team clears a planted-instruction alarm
- A PR waiting over five working days is flagged
- Weekly summary: open PRs by age, pages overdue, share verified, alarms

A small part of the graph, revealed step by step



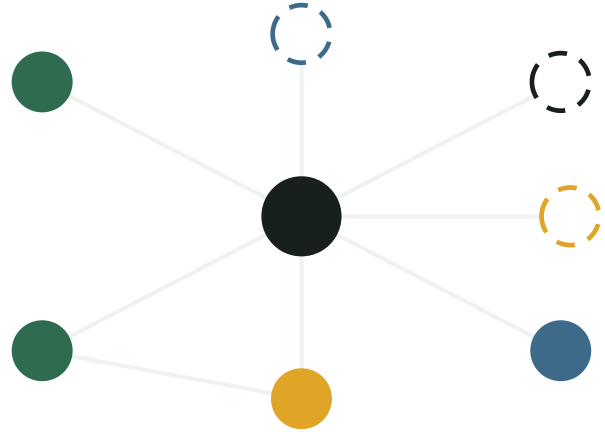
1. At session start

The repo's service page, and one line on where the feature stands. Nothing else.



2. As the stage needs it

Architecture: decisions governing the service, the interfaces of what it depends on, blocking standards.



3. On demand

Claude asks what's around a page, sees one line per neighbour, and opens only what it needs.

Full page loaded · One-line summary · Not loaded

Service · Decision · Feature · Standard

Each reveal is capped at about 2,000 tokens; past it, one-line summaries only.

Questions to refine together

1. **Length limits**
   Are 250, 400, 300 and 500 words right for decisions, services, features and standards?

2. **Reveal cap**
   Is about 2,000 tokens per reveal too tight or too loose?

3. **Engineers' lessons**
   Keep them inferred (team review), or trust them once code review passes?

4. **Re-verification**
   Are 75 days and five PRs per domain per week the right rhythm?

5. **What each stage reveals**
   For example, should Intake see past features at all?

6. **Standards outside the stages**
   How should they reach Claude when someone codes without the stage commands?

7. **Branch rules**
   Owner approval on some paths and none on others: who tests it on a real repo?

8. **Org owners**
   Who approves decisions that bind more than one domain, in practice?

Next in the design: seeding the first service pages with DeepWiki-Open.